

Week 5 -Threat Hunting Concept

Hypothesis-Driven Hunt for Remote PowerShell over WinRM (MITRE ATT&CK T1021.006)

Course: Cyber Threat Intelligence

Theme: Remote PowerShell Attacks via Windows Remote Management (WinRM) -MITRE ATT&CK T1021.006

Students: Aruzhan Abdulina, Kanasheva Darina

1. Assignment Recap and Overview

According to the course syllabus (Week 5 -Threat Hunting Concept), the practice session consists of two tasks:

- Build a hypothesis-driven hunting scenario (for example, suspicious PowerShell activity).
- Execute hunt queries in Splunk or ELK.

Recommended reading: the Microsoft Threat Hunting Guide and Phillip Smith, Practical Threat Hunting. The lecture covers hunting models (intel-driven and hypothesis-driven).

1.1 What this lab is about, in plain words

In Weeks 1–4 we studied one topic: attackers who use WinRM and PowerShell Remoting to run commands on other computers (technique T1021.006), as the ransomware group Storm-0501 did with the Evil-WinRM tool. Those weeks were about collecting and organising information. Week 5 asks a different question: **if such activity were happening in a network, could we actually find it in the logs?**

Threat hunting means a person actively searching the collected data for attacker behaviour that no alarm has reported yet. Instead of waiting for an alert, the analyst starts with an idea ("a hypothesis"), checks it against the data, and decides whether the idea is confirmed or rejected.

1.2 What we did

1. Formed a hypothesis from the Week 4 intelligence on Storm-0501.
2. Built a small lab: a Windows 11 virtual machine with WinRM enabled, Sysmon recording process and network activity, and Splunk Enterprise collecting those logs.
3. Generated known activity: discovery commands run locally (control) and remotely through WinRM (simulated "attack").
4. Ran five hunts in Splunk to see whether the remote activity can be separated from the local activity.
5. Evaluated the results, including a false positive, and listed the limits of the lab.

2. Threat Hunting and Hunting Models

Threat hunting is a proactive, analyst-led search through telemetry for malicious activity that existing detections have missed. It assumes that an attacker may already be inside the network.

The lecture distinguishes two models:

	Intel-driven hunting	Hypothesis-driven hunting
Starting point	A known indicator or report: a hash, IP address, domain or named campaign.	An assumption about how an attacker would behave in our environment.
What is searched	Indicators of compromise (IOCs) in the logs.	Behaviour (process relationships, command patterns, timing).
Strength	Fast and precise when the indicator is fresh.	Can find new or modified tools, because behaviour changes less than indicators.
Weakness	Misses anything not yet in a feed; indicators go stale.	Needs good data and more analyst effort.
Example in this lab	Hunt 5: search for the Embargo SHA-256 from Week 2.	Hunts 1–4: find remote commands through the parent process.

Model used. The main approach in this lab is **hypothesis-driven**. We also made one intel-driven pivot (Hunt 5), which shows how the two models complement each other. The generic hunting loop we followed is:

1. **Hypothesis:** state what we expect attacker activity to look like.
2. **Data:** decide which logs can prove or disprove it.
3. **Hunt:** write and run queries.
4. **Analyse:** separate true findings from normal activity and false positives.
5. **Conclude and improve:** confirm or reject the hypothesis, then turn what worked into a detection.

3. Hypothesis

3.1 Where the hypothesis comes from

The Week 4 analysis of the Microsoft Threat Intelligence reports showed that Storm-0501 used Evil-WinRM after it already held domain administrator rights. Over those sessions it ran discovery commands such as `sc query`, `quser` and `net`. That is intelligence about **behaviour**, not an indicator, which is why it suits a hypothesis-driven hunt.

3.2 Hypothesis statement

If an attacker uses WinRM for remote command execution (T1021.006), **then** the discovery commands they run will be started by `wsmprovhost.exe`, the WinRM host process, **whereas** the same commands typed in a local PowerShell window will be started by `powershell.exe`. Sysmon process-creation events (Event ID 1) should therefore let us tell remote activity from local activity.

The hypothesis is rejected if the parent process is the same in both cases, or if the remote commands leave no trace in Sysmon.

3.3 Testable predictions

#	Prediction	Tested in
P1	Commands run through Invoke-Command have the parent <code>wsmprovhost.exe</code> .	Hunt 1

#	Prediction	Tested in
P2	The same commands run locally have the parent powershell.exe, so the parent separates the two cases.	Hunt 2 (control)
P3	Several different discovery tools appear within a few minutes under one parent.	Hunt 3
P4	WinRM network connections (ports 5985/5986) appear at the same time as the remote commands.	Hunt 4
P5	The Embargo hash from Weeks 2–3 is not present on this host.	Hunt 5

3.4 ATT&CK techniques covered

Technique	Name	Command in the lab
T1021.006	Remote Services: Windows Remote Management	Invoke-Command -ComputerName localhost
T1059.001	Command and Scripting Interpreter: PowerShell	PowerShell script block executed remotely
T1033	System Owner/User Discovery	whoami
T1016	System Network Configuration Discovery	ipconfig /all
T1057	Process Discovery	tasklist
T1087.001	Account Discovery: Local Account	net user
T1082	System Information Discovery	systeminfo

Technique IDs should be double-checked on attack.mitre.org before submission.

4. Methodology

4.1 Lab design

Component	Role
Windows 11 VM (WinDev2407Eval, VirtualBox, 8 GB RAM)	The only machine. It plays both "attacker" and "victim", which is a simplified lab setup.
WinRM / PowerShell Remoting	Generates the remote activity of technique T1021.006.
Sysmon 15.20	Records process creation (Event ID 1), network connections (Event ID 3) and process termination (Event ID 5).
Splunk Enterprise 10.6.0.5	Collects the Sysmon log and lets us run the hunt queries.

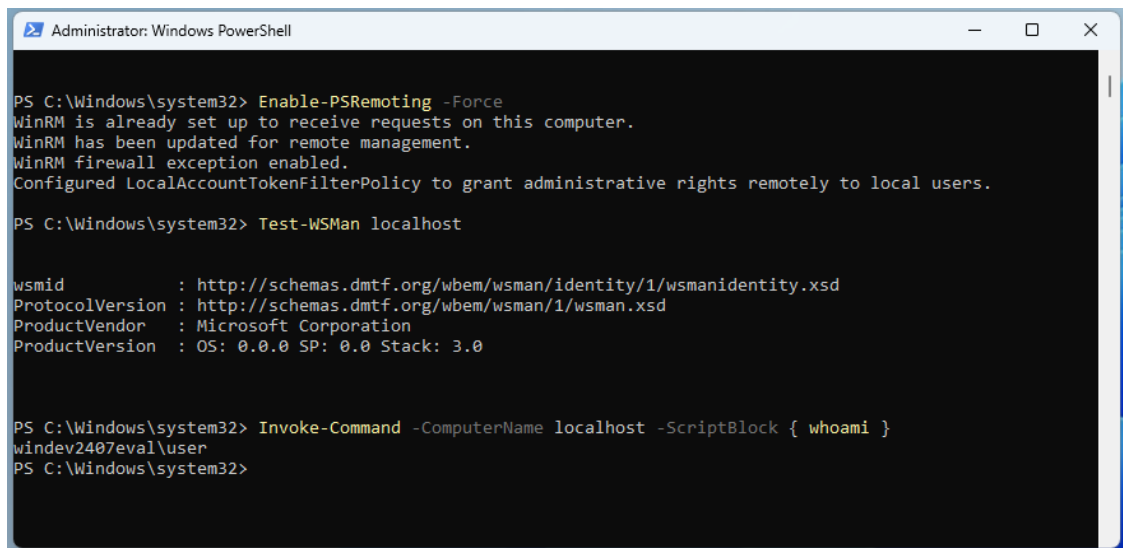
4.2 Experimental design

- **Control run:** whoami; ipconfig /all; tasklist typed locally in PowerShell. This shows what normal local activity looks like.

- **Simulated attack run:** Invoke-Command -ComputerName localhost -ScriptBlock { whoami; ipconfig /all; tasklist; net user; systeminfo }. These are the same kinds of discovery commands Storm-0501 ran over Evil-WinRM.
- **Comparison:** hunt queries then compare parent processes, timing and network data for the two runs. Having a control is what makes the result meaningful: it shows whether the signal is specific to WinRM or just a feature of PowerShell in general.

5. Implementation, Step by Step

5.1 Enabling WinRM



```
PS C:\Windows\system32> Enable-PSRemoting -Force
WinRM is already set up to receive requests on this computer.
WinRM has been updated for remote management.
WinRM firewall exception enabled.
Configured LocalAccountTokenFilterPolicy to grant administrative rights remotely to local users.

PS C:\Windows\system32> Test-WSMan localhost

wsmid      : http://schemas.dmtf.org/wbem/wsman/identity/1/wsmanidentity.xsd
ProtocolVersion : http://schemas.dmtf.org/wbem/wsman/1/wsman.xsd
ProductVendor   : Microsoft Corporation
ProductVersion  : OS: 0.0.0 SP: 0.0 Stack: 3.0

PS C:\Windows\system32> Invoke-Command -ComputerName localhost -ScriptBlock { whoami }
windev2407eval\user
PS C:\Windows\system32>
```

Figure 1. WinRM enabled and verified on the lab VM (Windows 11, WinDev2407Eval).

What it shows. Three commands in an administrator PowerShell window. Enable-PSRemoting -Force reports that WinRM is set up, the firewall exception is enabled and LocalAccountTokenFilterPolicy is configured. Test-WSMan localhost shows the service answers (vendor: Microsoft Corporation). Invoke-Command -ComputerName localhost -ScriptBlock { whoami } runs a command through a WinRM session and returns windev2407eval\user.

Why it matters. The VM can now produce the same kind of activity as technique T1021.006. LocalAccountTokenFilterPolicy gives local accounts full administrative rights over remote connections. It is convenient in a lab but weakens security in a real network, so it is worth monitoring.

5.2 Checking that Sysmon sees the activity

```
WinDev2407Eval [Pafoaer] - Oracle VirtualBox
Administrator: Windows PowerShell

windev2407eval\user
PS C:\Windows\system32>
PS C:\Windows\system32>
PS C:\Windows\system32> Get-Service | Where-Object { $_.Name -like "Sysmon*" }

Status Name          DisplayName
-----
Running Sysmon64      Sysmon64

PS C:\Windows\system32> Sysmon64.exe -c

System Monitor v15.20 - System activity monitor
By Mark Russinovich and Thomas Garnier
Copyright (C) 2014-2026 Microsoft Corporation
Using libxml2. libxml2 is Copyright (C) 1998-2012 Daniel Veillard. All Rights Reserved.
sysinternals - www.sysinternals.com

Current configuration:
- Service name: Sysmon64
- Driver name: SysmonDrv
- Config file: C:\Users\User\Downloads\Sysmon\sysmon64.exe -i -n
- HashingAlgorithms: SHA256
- Network connection: enabled
- Archive Directory: disabled
- Image loading: disabled
- CRL checking: enabled
- DNS lookup: enabled

No rules installed
PS C:\Windows\system32> Sysmon64.exe -c
```

Figure 2. Sysmon64 service running and its current configuration.

What it shows. Get-Service shows the Sysmon64 service in the Running state. Sysmon64.exe -c prints the configuration: SHA-256 hashing, network connection logging enabled, image loading disabled, and "No rules installed".

Why it matters. Sysmon is the data source for every hunt. With no custom rule set, it records its default events, which is enough for process creation and network connections but is not a tuned production configuration (see Limitations).

```
WinDev2407Eval [Pafoaer] - Oracle VirtualBox
Administrator: Windows PowerShell

No rules installed
PS C:\Windows\system32> whoami
windev2407eval\user
PS C:\Windows\system32> Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 5 | Format-List TimeCreated, Id, Message

TimeCreated : 10/3/2026 9:02:57 AM
Id           : 3
Message      : Network connection detected:
               RuleName: -
               UtcTime: 2026-10-03 16:03:51.080
               ProcessGuid: {54d14d2f-2016-6ac1-a701-000000002d00}
               ProcessId: 5108
               Image: c:\Users\User\AppData\Local\Microsoft\OneDrive\OneDrive.exe
               User: WINDEV2407EVAL\User
               Protocol: tcp
               Initiated: true
               SourceIsIpv6: false
               SourceIp: 192.168.0.6
               SourceHostname: WinDev2407Eval.localhost
               SourcePort: 50135
               SourcePortName: -
               DestinationIsIpv6: false
               DestinationIp: 20.184.175.0
               DestinationHostname: -
               DestinationPort: 443
               DestinationPortName: https

TimeCreated : 10/3/2026 9:02:55 AM
Id           : 3
Message      : Network connection detected:
               RuleName: -
               UtcTime: 2026-10-03 16:03:50.581
               ProcessGuid: {54d14d2f-19a9-6ac1-3200-000000002d00}
               ProcessId: 2412
               Image: C:\Windows\System32\svchost.exe
               User: NT AUTHORITY\NETWORK SERVICE
               Protocol: udp
```

Figure 3. Local `whoami` (control) followed by the latest Sysmon events.

What it shows. After running whoami locally, Get-WinEvent reads the Sysmon log. The most recent entries are Event ID 3 (network connection) records, for example OneDrive.exe and svchost.exe connecting out over HTTPS and UDP.

Why it matters. It confirms Sysmon writes to its Operational log in real time, and it shows how much ordinary background activity the log contains. A hunter has to find the attack inside that noise.

```
PS C:\Windows\system32> Invoke-Command -ComputerName localhost -ScriptBlock { whoami }
windev2407eval\User
PS C:\Windows\system32> Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 10 | Where-Object { $_.Message -like "wsmprovhost*" } | Format-List TimeCreated, Id, Message

TimeCreated : 10/3/2026 9:05:42 AM
Id           : 5
Message      : Process terminated:
               RuleName:
              UtcTime: 2026-10-03 16:05:42.893
               ProcessGuid: {5d1d42f-2742-6ac1-8302-000000002d00}
               ProcessId: 2860
               Image: C:\Windows\System32\wsmprovhost.exe
               User: WINDEV2407EVAL\User
PS C:\Windows\system32>
```

Figure 4. Remote `whoami` through WinRM (simulated attack) and the matching Sysmon trace.

What it shows. Invoke-Command -ComputerName localhost -ScriptBlock { whoami } is run, and the Sysmon log is then filtered for the text wsmprovhost. The result is an Event ID 5 record (process terminated) for C:\Windows\System32\wsmprovhost.exe under the user WINDEV2407EVAL\User.

Why it matters. This is the first evidence supporting the hypothesis: a remote WinRM command makes Windows start the wsmprovhost.exe process, and Sysmon records it.

5.3 Installing Splunk and connecting the Sysmon log

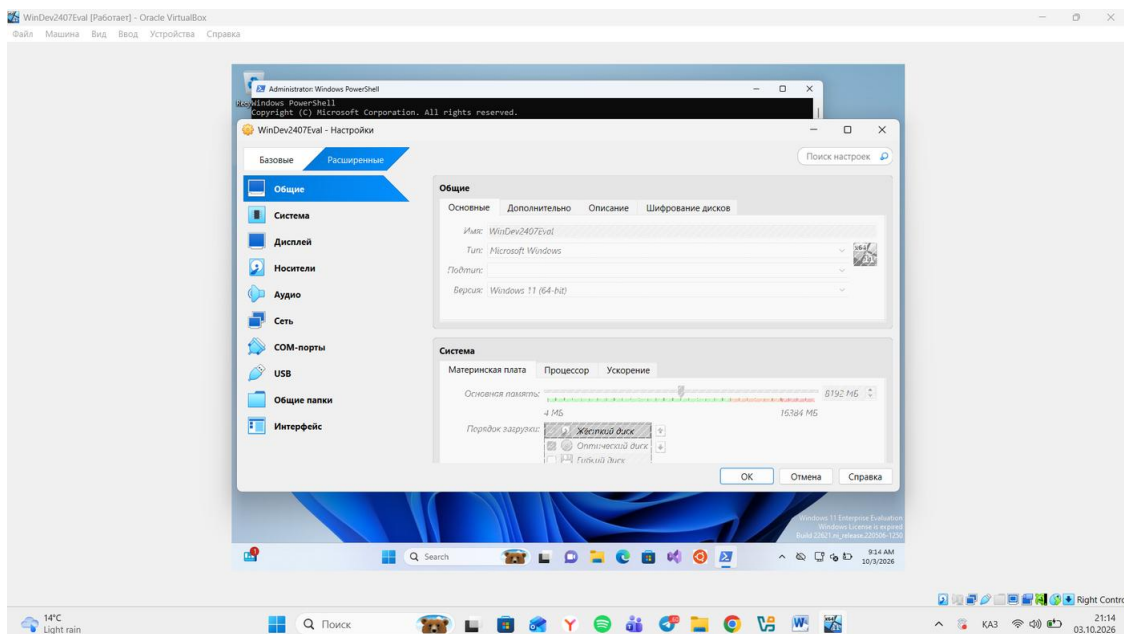


Figure 5. VirtualBox settings of the lab VM before installing Splunk.

What it shows. VM name WinDev2407Eval, type Windows 11 (64-bit), 8192 MB of memory.

Why it matters. Splunk Enterprise is resource-hungry, so we checked that the VM had enough memory before installing it.

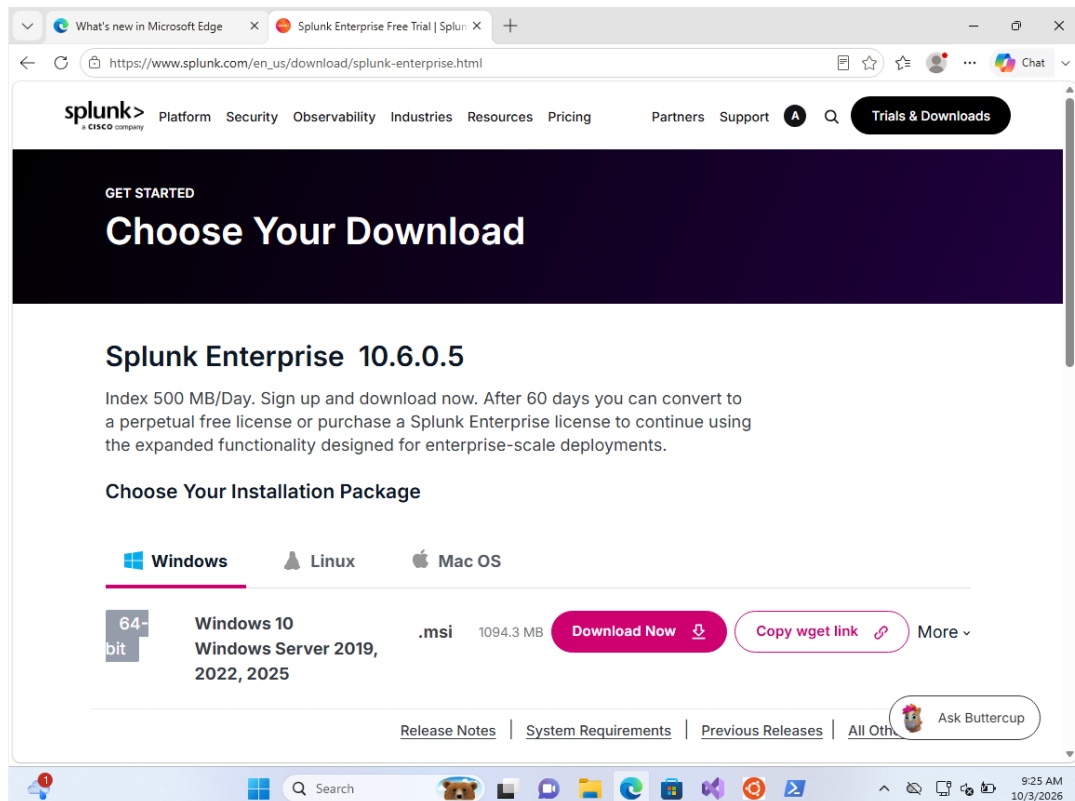


Figure 6. Splunk Enterprise 10.6.0.5 download page (Windows 64-bit .msi).

What it shows. The Splunk download page offering version 10.6.0.5 for Windows. The page states it indexes up to 500 MB per day and can be converted to a free licence after 60 days.

Why it matters. This is the SIEM (log search platform) used for the hunts; the free trial is sufficient for a lab.

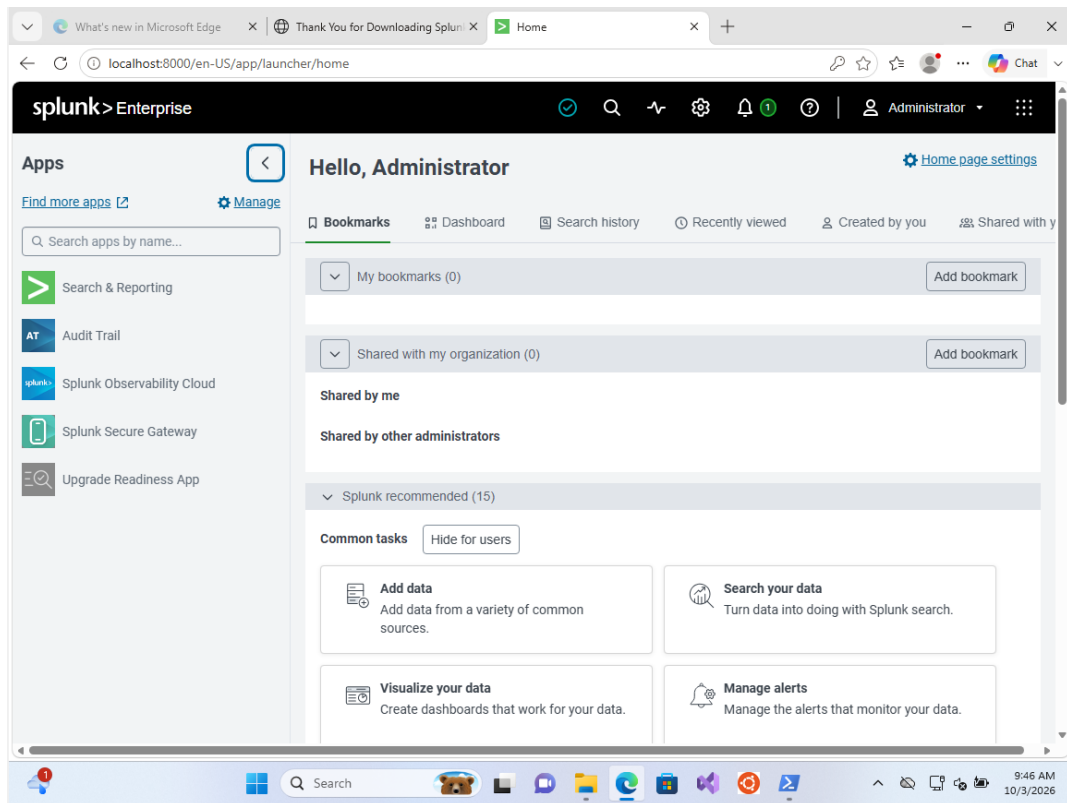


Figure 7. Splunk Enterprise home page after installation.

What it shows. The Splunk web interface running on localhost:8000, signed in as Administrator, with the Search & Reporting app available.

Why it matters. It confirms the installation worked and Splunk is reachable from the VM.

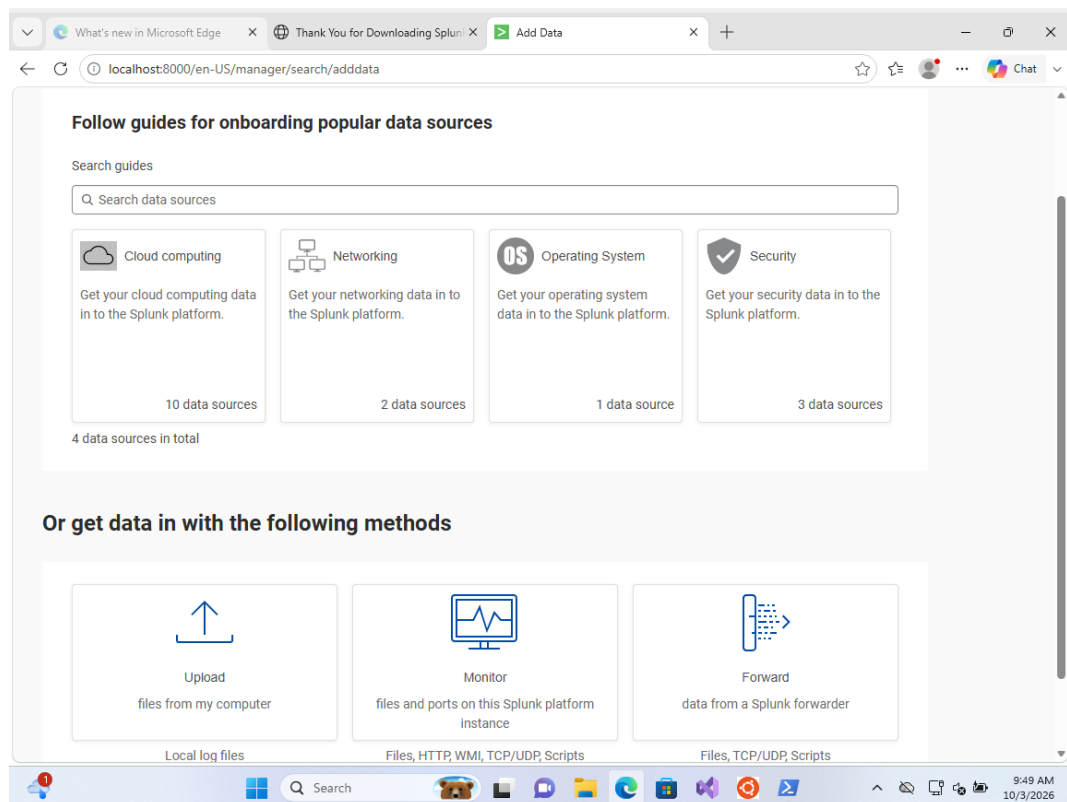


Figure 8. Splunk "Add Data" page.

What it shows. Splunk's data onboarding screen with the options Upload, Monitor and Forward.

Why it matters. Splunk is empty until data is added; this is where log sources are connected.

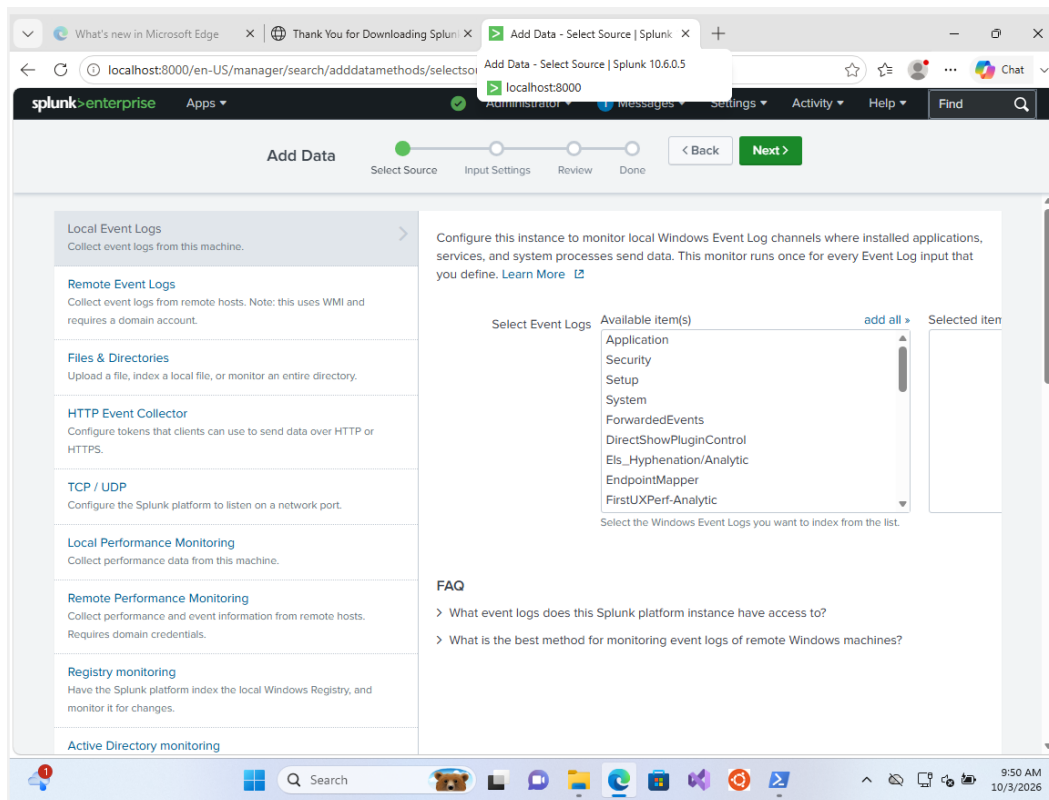


Figure 9. Add Data wizard: "Local Event Logs" source selection.

What it shows. The wizard for collecting Windows event logs from this machine, listing channels such as Application, Security, Setup and System.

Why it matters. It shows that Windows logs can be added through the interface. For the Sysmon channel we then edited the configuration file directly (next figure).

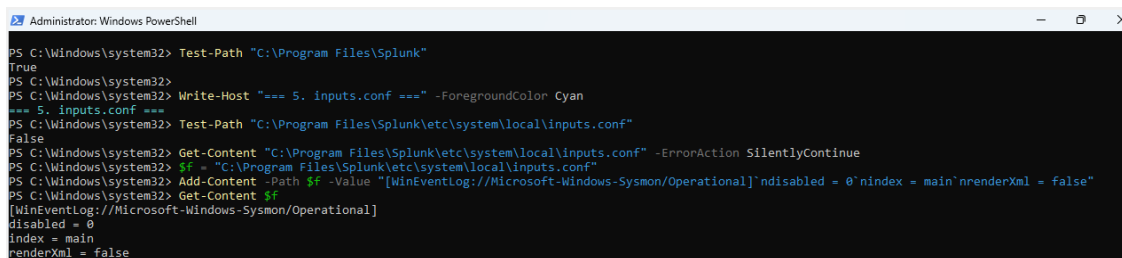


Figure 10. Sysmon connected to Splunk through `inputs.conf`.

What it shows. Test-Path confirms Splunk is installed and that inputs.conf did not exist yet. A stanza is then written for [WinEventLog://Microsoft-Windows-Sysmon/Operational] with disabled = 0, index = main and renderXml = false, and Get-Content displays the result.

Why it matters. These lines tell Splunk to read the Sysmon Operational log and store it in the main index. Plain-text rendering (renderXml = false) makes fields such as Image, ParentImage and CommandLine easy to search. Splunk must be restarted for this change to take effect.

5.4 Generating the activity

```

Administrator: Windows PowerShell
PS C:\Windows\system32>
PS C:\Windows\system32>
PS C:\Windows\system32> Get-Service splunkd

Status Name          DisplayName
-----
Running Splunkd      splunkd

PS C:\Windows\system32> whoami; ipconfig /all; tasklist
WinDev2407Eval\user

Windows IP Configuration

Host Name . . . . . : WinDev2407Eval
Primary Dns Suffix . . . . . :
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No
DNS Suffix Search List. . . . . : localhost

Ethernet adapter Ethernet:

Connection-specific DNS Suffix . : localhost
Description . . . . . : Intel(R) PRO/1000 MT Desktop Adapter
Physical Address. . . . . : 08-00-27-C3-FF-56
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80::cbb8:3f6a:9692:7244%12(Preferred)
IPv4 Address. . . . . : 192.168.0.6(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : Sunday, October 4, 2026 3:26:48 PM
Lease Expires . . . . . : Monday, October 5, 2026 3:26:45 PM
Default Gateway . . . . . : fe80::1%12
                          192.168.0.1
DHCP Server . . . . . : 192.168.0.1
DHCPv6 IAID . . . . . : 101187623
DHCPv6 Client DUID. . . . . : 00-01-00-01-31-BA-8E-24-08-00-27-C3-FF-56
DNS Servers . . . . . : fe80::1%12
                          5.34.34.5

```

Figure 11. Local run (control): ``whoami; ipconfig /all; tasklist``.

What it shows. After confirming the splunkd service is running, the three discovery commands are typed directly in PowerShell. The output shows the host WinDev2407Eval, IP address 192.168.0.6 and the process list.

Why it matters. This is the control. These commands are started by the local PowerShell process, and Hunt 2 will use this run to show the difference from the remote run.

```

splunk-optimize.exe          1964 Services          0      1,888 K
PS C:\Windows\system32> Invoke-Command -ComputerName localhost -ScriptBlock { whoami; ipconfig /all; tasklist; net user; systeminfo }
WinDev2407Eval\user

Windows IP Configuration

Host Name . . . . . : WinDev2407Eval
Primary Dns Suffix . . . . . :
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No
DNS Suffix Search List. . . . . : localhost

Ethernet adapter Ethernet:

Connection-specific DNS Suffix . : localhost
Description . . . . . : Intel(R) PRO/1000 MT Desktop Adapter
Physical Address. . . . . : 08-00-27-C3-FF-56
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80::cbb8:3f6a:9692:7244%12(Preferred)
IPv4 Address. . . . . : 192.168.0.6(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : Sunday, October 4, 2026 3:26:47 PM
Lease Expires . . . . . : Monday, October 5, 2026 3:26:42 PM
Default Gateway . . . . . : fe80::1%12
                          192.168.0.1
DHCP Server . . . . . : 192.168.0.1
DHCPv6 IAID . . . . . : 101187623
DHCPv6 Client DUID. . . . . : 00-01-00-01-31-BA-8E-24-08-00-27-C3-FF-56
DNS Servers . . . . . : fe80::1%12
                          5.34.34.5
                          37.99.99.37
                          192.168.0.1
NetBIOS over Tcpip. . . . . : Enabled

Image Name                      PID Session Name        Session#    Mem Usage
-----
System Idle Process             0 Services              0            8 K

```

Figure 12. Remote run (simulated attack): five discovery commands through ``Invoke-Command``.

What it shows. Invoke-Command -ComputerName localhost -ScriptBlock { whoami; ipconfig /all; tasklist; net user; systeminfo }. The top of the window still shows the end of the earlier local tasklist output.

Why it matters. This is the activity we try to detect. The commands imitate the discovery steps Storm-0501 performed over Evil-WinRM: user, network, processes, accounts and system information.

5.5 Verifying that Splunk receives the data

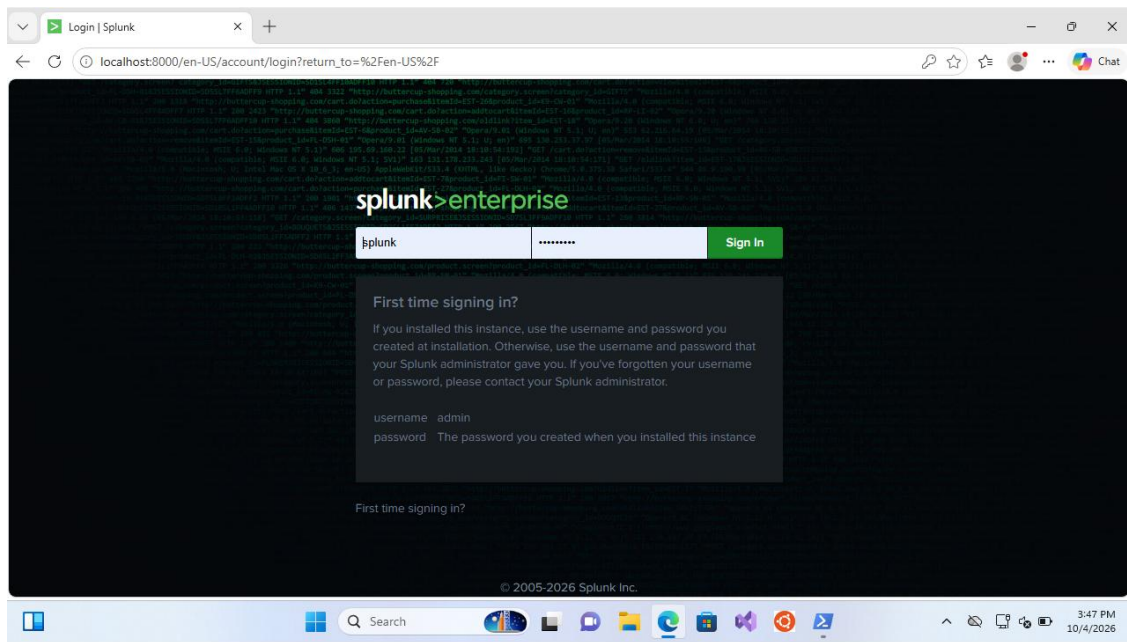


Figure 13. Splunk login page.

What it shows. The Splunk sign-in screen on localhost:8000.

Why it matters. It documents the point where the analyst logs in to start searching. (The password field is masked.)

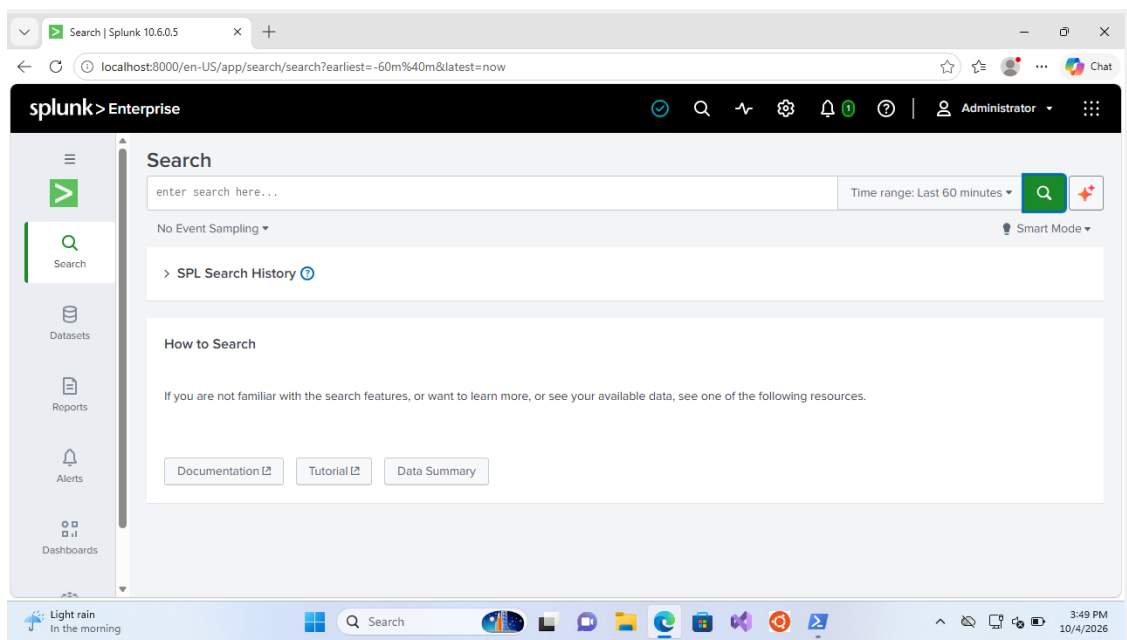


Figure 14. Empty Splunk search page.

What it shows. The Search & Reporting app with an empty query box and the time range set to the last 60 minutes.

Why it matters. The starting point for all searches.

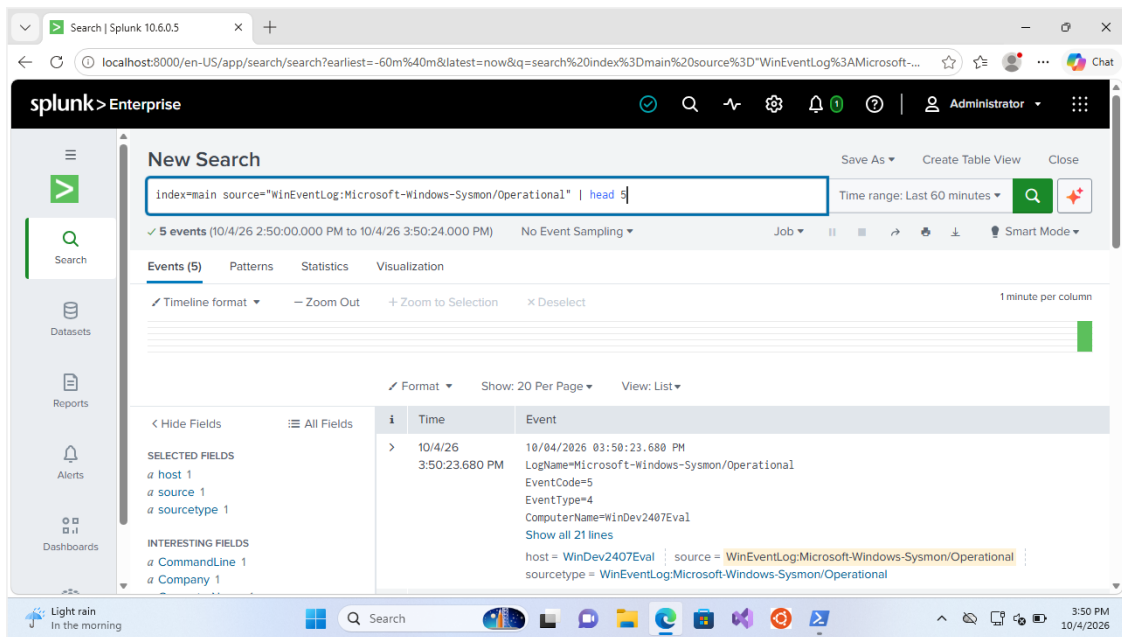


Figure 15. First check: ``index=main source="WinEventLog:Microsoft-Windows-Sysmon/Operational" | head 5``.

What it shows. The query returns 5 events from the Sysmon log, for host WinDev2407Eval. The first is EventCode 5 (process terminated).

Why it matters. It proves the pipeline Sysmon → Splunk works and that events are searchable by source name.

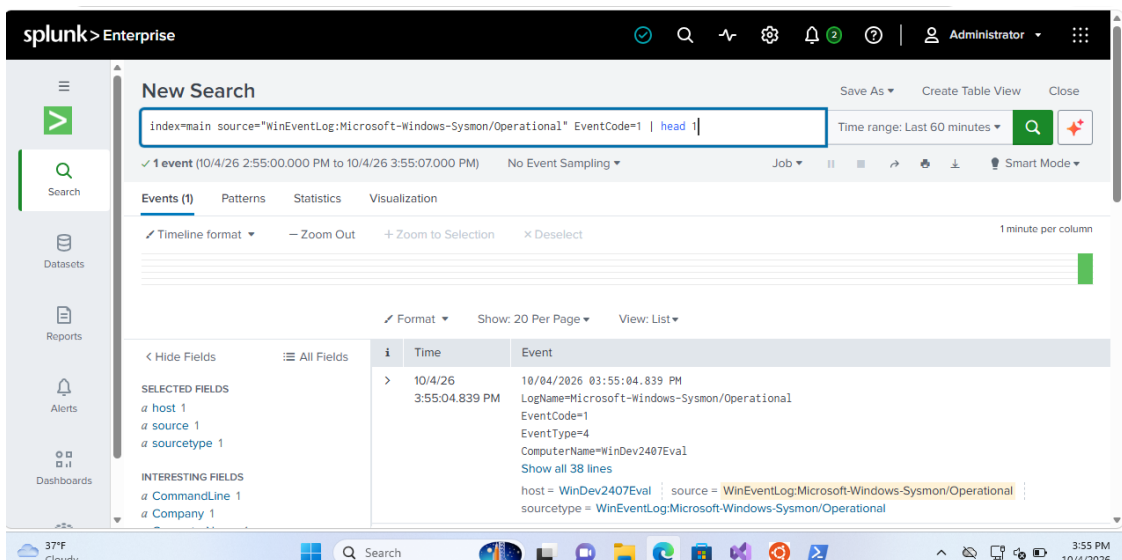


Figure 16. Process-creation check: ``EventCode=1 | head 1`` (last 60 minutes).

What it shows. One EventCode 1 (process creation) event, expanded to its extracted fields.

Why it matters. Event ID 1 is the key data type for hypothesis-driven hunting here, because it records which process started which.

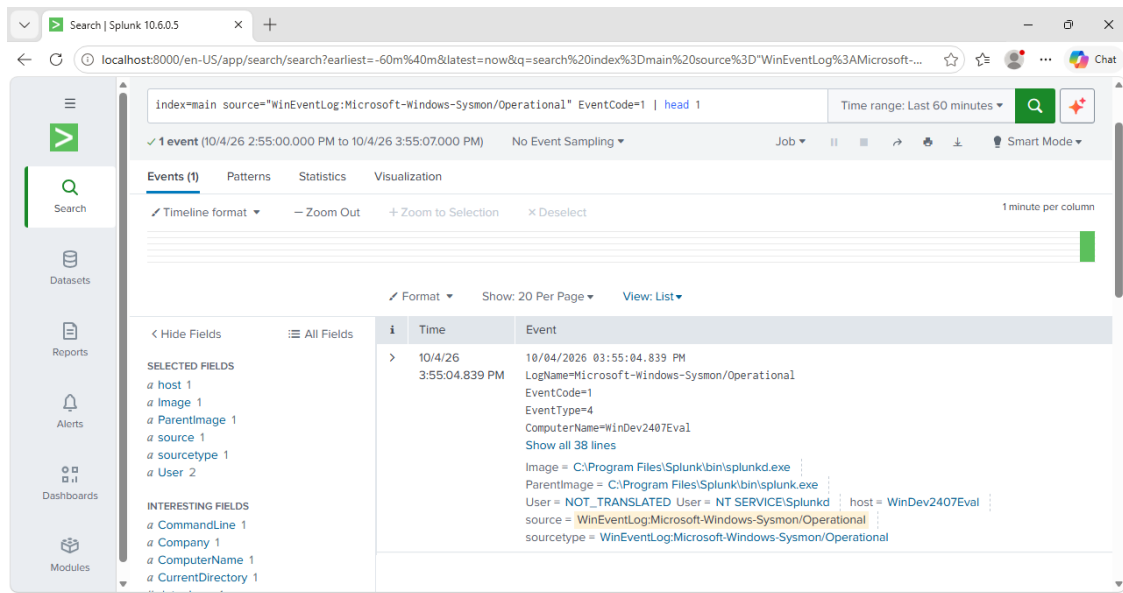


Figure 17. Process-creation event with extracted fields (`Image`, `ParentImage`, `CommandLine`, `User`).

What it shows. The latest process-creation event is Splunk's own splunkd.exe, started by splunk.exe under NT SERVICE\Splunkd. The field list shows Image, ParentImage, CommandLine, User and others.

Why it matters. Splunk has extracted the exact fields the hunts depend on. It also shows that the lab machine generates background noise of its own, including Splunk.

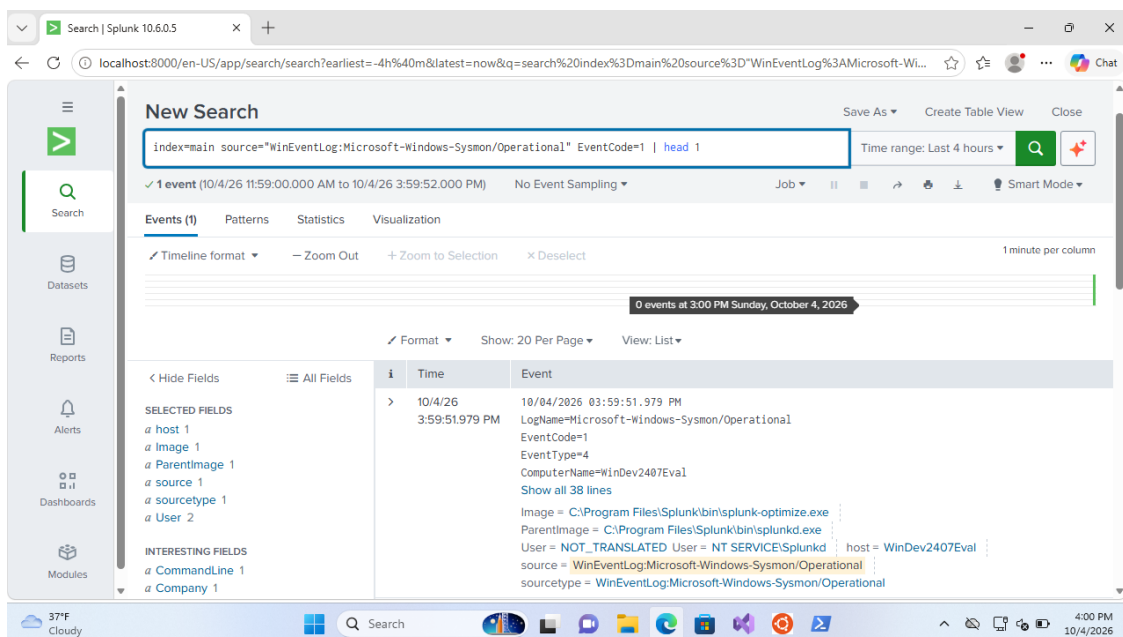


Figure 18. Same check with a 4-hour time range.

What it shows. The same EventCode=1 | head 1 query over the last 4 hours, returning splunk-optimize.exe started by splunkd.exe.

Why it matters. The 4-hour window is the one used for all following hunts, so it covers both the control and the remote run.

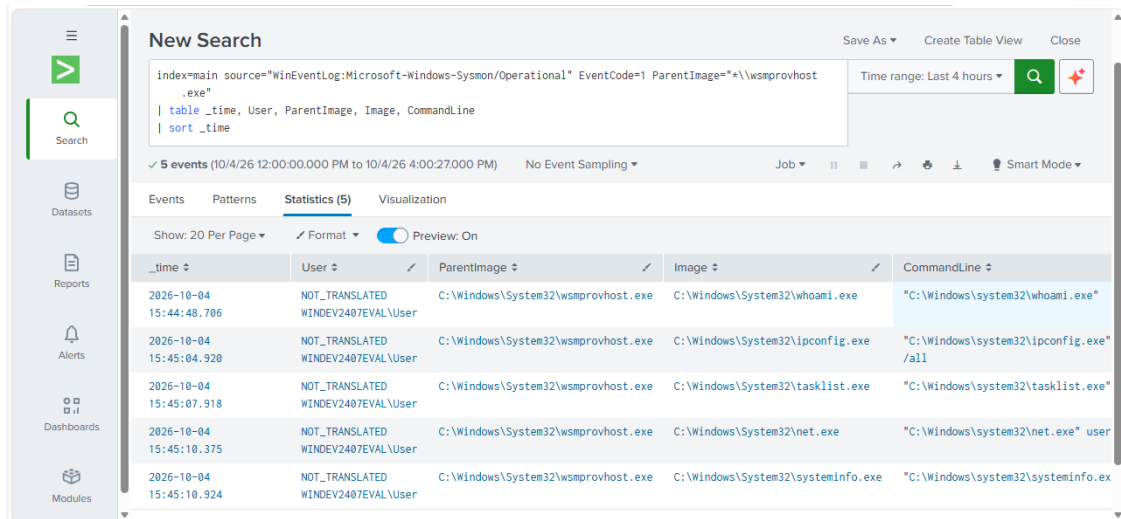
6. Hunts and Results

All hunts use index=main source="WinEventLog:Microsoft-Windows-Sysmon/Operational" over the last 4 hours.

6.1 Hunt 1 -Commands started by the WinRM host process (tests P1)

Question: which processes have wsmprovhost.exe as their parent?

```
index=main source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
ParentImage="*\\wsmprovhost.exe"
| table _time, User, ParentImage, Image, CommandLine
| sort _time
```



_time	User	ParentImage	Image	CommandLine
2026-10-04 15:44:48.706	NOT_TRANSLATED WINDEV2407EVAL\\User	C:\\Windows\\System32\\wsmprovhost.exe	C:\\Windows\\System32\\whoami.exe	"C:\\Windows\\system32\\whoami.exe"
2026-10-04 15:45:04.920	NOT_TRANSLATED WINDEV2407EVAL\\User	C:\\Windows\\System32\\wsmprovhost.exe	C:\\Windows\\System32\\ipconfig.exe	"C:\\Windows\\system32\\ipconfig.exe" /all
2026-10-04 15:45:07.918	NOT_TRANSLATED WINDEV2407EVAL\\User	C:\\Windows\\System32\\wsmprovhost.exe	C:\\Windows\\System32\\tasklist.exe	"C:\\Windows\\system32\\tasklist.exe"
2026-10-04 15:45:10.375	NOT_TRANSLATED WINDEV2407EVAL\\User	C:\\Windows\\System32\\wsmprovhost.exe	C:\\Windows\\System32\\net.exe	"C:\\Windows\\system32\\net.exe" user
2026-10-04 15:45:10.924	NOT_TRANSLATED WINDEV2407EVAL\\User	C:\\Windows\\System32\\wsmprovhost.exe	C:\\Windows\\System32\\systeminfo.exe	"C:\\Windows\\system32\\systeminfo.exe"

Figure 19. Hunt 1 results: 5 events with parent `wsmprovhost.exe`.

Result. Five events, all at about 15:44–15:45 on 2026-10-04: whoami.exe, ipconfig.exe /all, tasklist.exe, net.exe user and systeminfo.exe. Every one has C:\\Windows\\System32\\wsmprovhost.exe as the parent and runs as WINDEV2407EVAL\\User.

Interpretation. This is exactly the remote command set we sent. **P1 is confirmed:** commands arriving over WinRM are children of wsmprovhost.exe. In a real network, a burst of discovery tools under this parent from an unexpected account or at an unexpected time would be a strong lead.

6.2 Hunt 2 -Control comparison (tests P2)

Question: are the same tools started differently when run locally?

```
index=main source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
| search Image IN ("*\\whoami.exe", "*\\ipconfig.exe", "*\\tasklist.exe",
                  "*\\net.exe", "*\\net1.exe", "*\\systeminfo.exe")
| table _time, User, ParentImage, Image
| sort _time
```

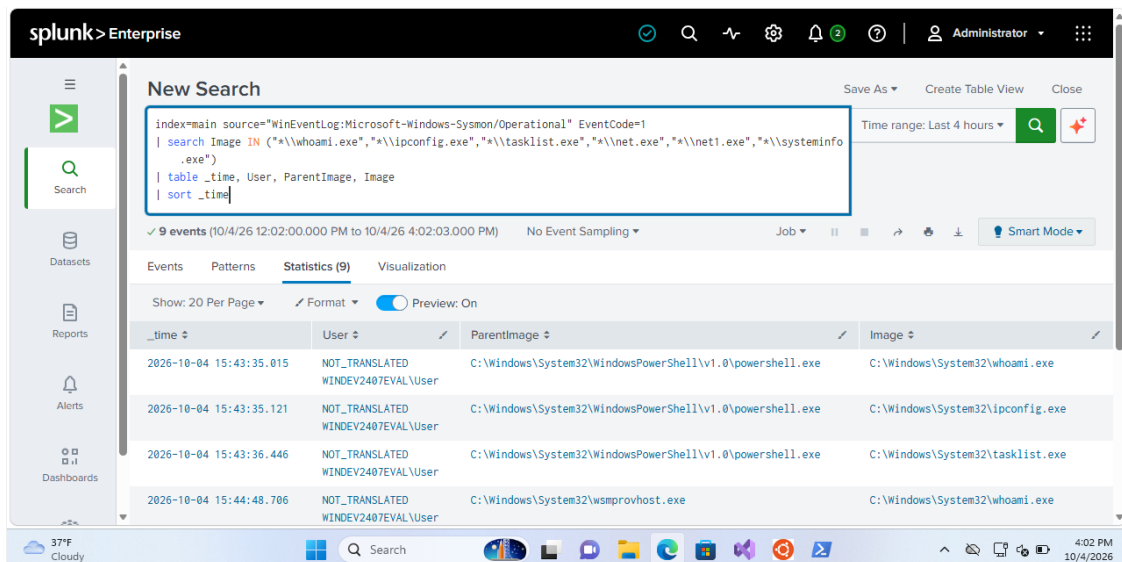


Figure 20. Hunt 2 query and first rows of the result (9 events).

What it shows. The query filters for the six discovery tools regardless of parent. The first rows (15:43) show whoami.exe, ipconfig.exe and tasklist.exe with the parent C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe, followed by a whoami.exe at 15:44:48 with the parent wsmprovhost.exe.

The screenshot shows the full result table for the search query. The table has the following columns: **_time**, **User**, **ParentImage**, and **Image**. The results are as follows:

_time	User	ParentImage	Image
2026-10-04 15:43:35.015	NOT_TRANSLATED WINDEV2407EVAL\User	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	C:\Windows\System32\whoami.exe
2026-10-04 15:43:35.121	NOT_TRANSLATED WINDEV2407EVAL\User	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	C:\Windows\System32\ipconfig.exe
2026-10-04 15:43:36.446	NOT_TRANSLATED WINDEV2407EVAL\User	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	C:\Windows\System32\tasklist.exe
2026-10-04 15:44:48.706	NOT_TRANSLATED WINDEV2407EVAL\User	C:\Windows\System32\wsmprovhost.exe	C:\Windows\System32\whoami.exe
2026-10-04 15:45:04.920	NOT_TRANSLATED WINDEV2407EVAL\User	C:\Windows\System32\wsmprovhost.exe	C:\Windows\System32\ipconfig.exe
2026-10-04 15:45:07.918	NOT_TRANSLATED WINDEV2407EVAL\User	C:\Windows\System32\wsmprovhost.exe	C:\Windows\System32\tasklist.exe
2026-10-04 15:45:10.375	NOT_TRANSLATED WINDEV2407EVAL\User	C:\Windows\System32\wsmprovhost.exe	C:\Windows\System32\net.exe
2026-10-04 15:45:10.806	NOT_TRANSLATED WINDEV2407EVAL\User	C:\Windows\System32\net.exe	C:\Windows\System32\net1.exe
2026-10-04 15:45:10.924	NOT_TRANSLATED WINDEV2407EVAL\User	C:\Windows\System32\wsmprovhost.exe	C:\Windows\System32\systeminfo.exe

Figure 21. Hunt 2: full result table (9 events).

Result. Three local events have the parent powershell.exe (15:43). Five remote events have the parent wsmprovhost.exe (15:44–15:45). The ninth, net1.exe, has the parent net.exe, which is normal because net launches net1 internally.

Interpretation. P2 is confirmed. The identical commands differ only by their parent process, so the parent separates remote from local activity. This is the core finding of the hunt and the reason a control was included: it shows the signal is specific to WinRM and not to PowerShell in general.

6.3 Hunt 3 -Clustering of discovery tools (tests P3)

Question: can we detect the pattern of several different discovery tools run close together by one parent?


```

index=main source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
| search Image IN ("*\\whoami.exe", "*\\ipconfig.exe", "*\\tasklist.exe",
                  "*\\net.exe", "*\\net1.exe", "*\\systeminfo.exe")
| eval tool=lower(replace(Image, ".*\\", ""))
| bin _time span=5m
| stats dc(tool) as distinct_tools values(tool) as tools by _time, User, ParentImage
| where distinct_tools >= 3

```

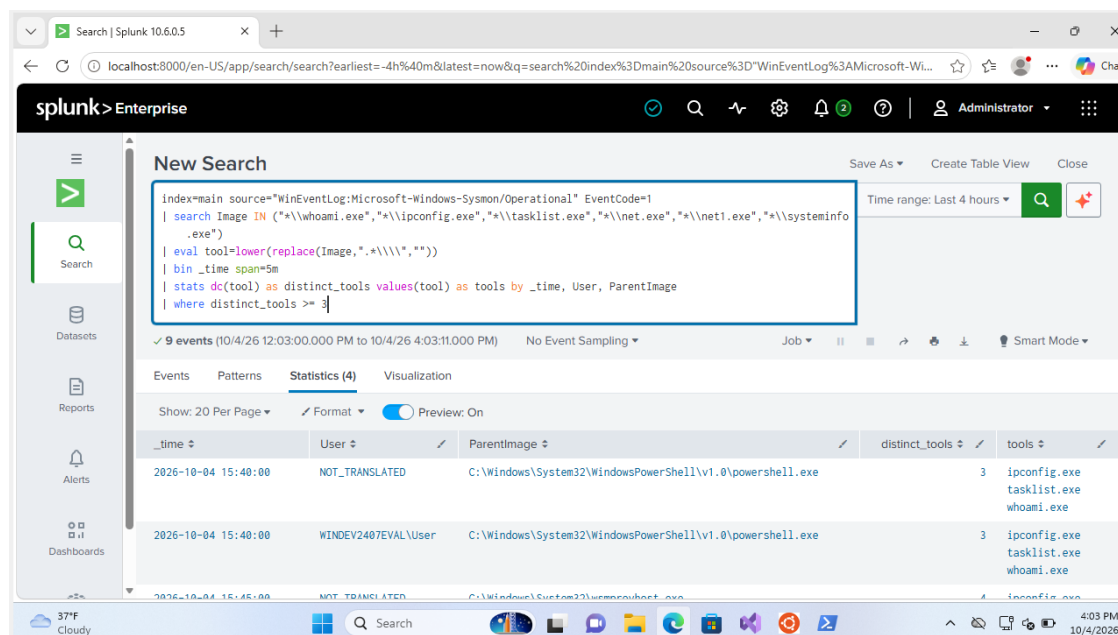


Figure 22. Hunt 3 query and the start of the result.

What it shows. The query groups events into 5-minute windows, counts distinct tools per parent process, and keeps groups with at least three tools.

Events	Patterns	Statistics (4)	Visualization
Show: 20 Per Page ▾ Format ▾ Preview: On			
_time ▾	User ▾	ParentImage ▾	distinct_tools ▾ tools ▾
2026-10-04 15:40:00	NOT_TRANSLATED	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	3 ipconfig.exe tasklist.exe whoami.exe
2026-10-04 15:40:00	WINDEV2407EVAL\User	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	3 ipconfig.exe tasklist.exe whoami.exe
2026-10-04 15:45:00	NOT_TRANSLATED	C:\Windows\System32\wsmprovhost.exe	4 ipconfig.exe net.exe systeminfo.exe tasklist.exe
2026-10-04 15:45:00	WINDEV2407EVAL\User	C:\Windows\System32\wsmprovhost.exe	4 ipconfig.exe net.exe systeminfo.exe tasklist.exe

Figure 23. Hunt 3: result table (4 rows).

Result. Two clusters were found: the local run (parent powershell.exe, 3 tools in the 15:40 window) and the remote run (parent wsmprovhost.exe, 4 tools in the 15:45 window). Each appears twice because the User field holds two values (NOT_TRANSLATED and WINDEV2407EVAL\User).

Interpretation. P3 is partly confirmed, and this hunt produced a false positive. The clustering rule also flags the harmless local control run, so it cannot be used alone. A usable detection would have to combine the cluster with the parent wsmprovhost.exe, as in Hunt 1.

Two details worth noting. First, the remote run shows four tools instead of five because the 5-minute window split it: whoami ran at 15:44:48 and fell into the earlier 15:40 window. Second, the duplicate rows could be removed by grouping on a single value of User. Both points show how the choice of time window and field handling affects a hunt.

6.4 Hunt 4 -WinRM network connections (tests P4)

Question: is the remote activity visible at network level, on the WinRM ports?

```
index=main source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=3  
(DestinationPort=5985 OR DestinationPort=5986)  
| head 20
```

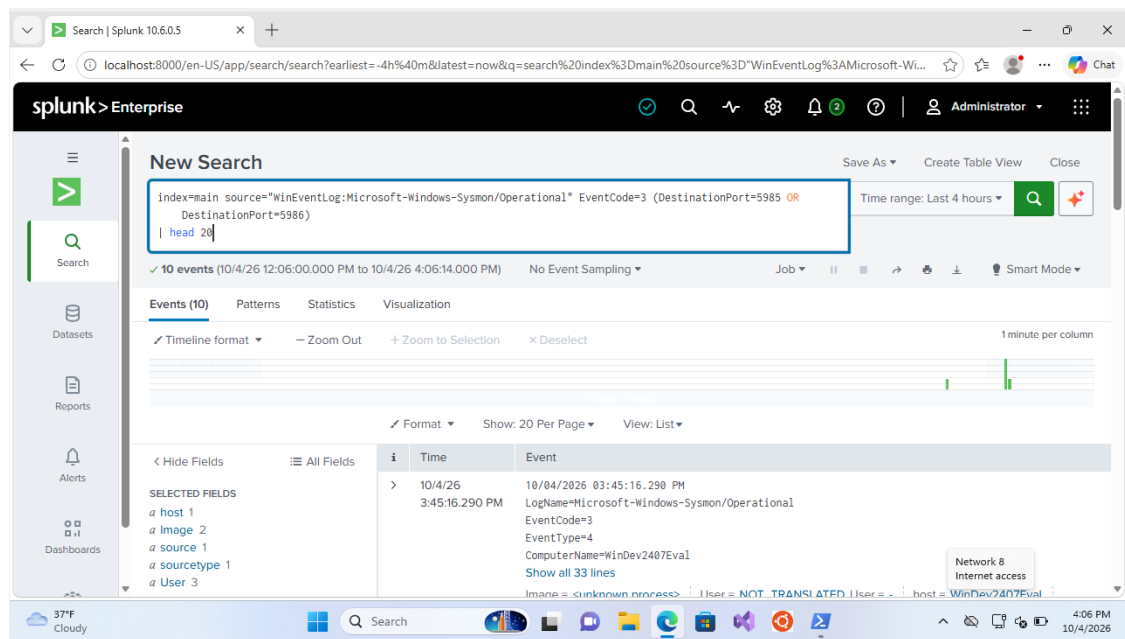


Figure 24. Hunt 4 query and result: 10 network events on ports 5985/5986.

What it shows. Ten Sysmon Event ID 3 records. The timeline at the top shows the events cluster in two short bursts.

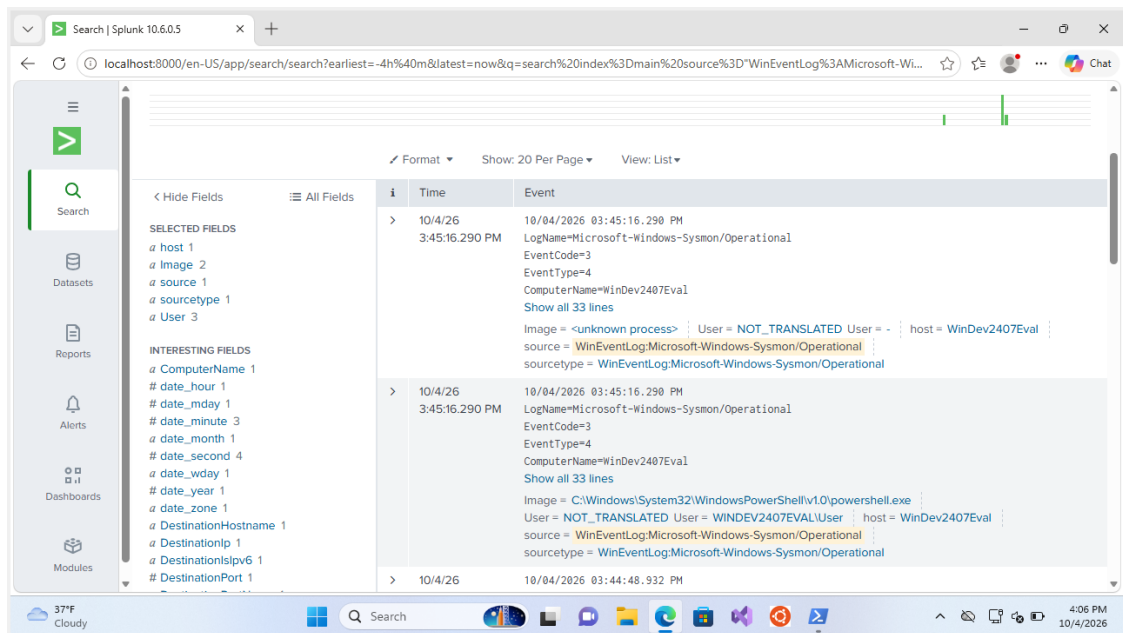


Figure 25. Hunt 4: expanded events.

Result. The events are timed 15:44:48–15:45:16, the same window as the remote commands. Some show the image powershell.exe and others show <unknown process>.

Interpretation. P4 is confirmed: WinRM traffic is visible and lines up in time with the remote commands. The <unknown process> entries are probably the receiving side of the connection, which Sysmon cannot attribute to a normal process. Because attacker and victim are the same machine, the connection is a loopback and gives no source/destination host pair; in a real network this hunt would reveal which machine connected to which.

6.5 Hunt 5 -Intel-driven pivot: the Embargo hash (tests P5)

Question: is the Embargo sample from Weeks 2–3 present on this host?

```
index=main source="WinEventLog:Microsoft-Windows-Sysmon/Operational"
"F4EC2DBA070A9ECD1DEE342329B0CD7CAA406CD4995C41B9AAB9AEA9B73CD06D"
```

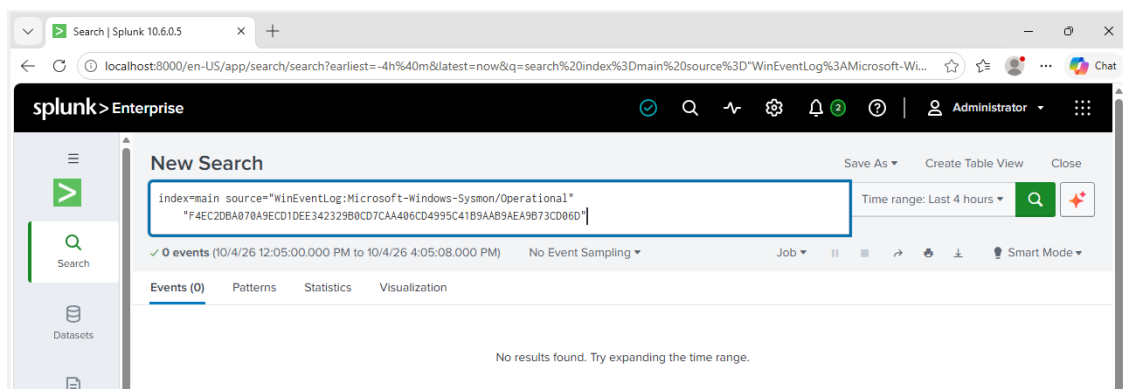


Figure 26. Hunt 5: search for the Week 2 SHA-256 returns 0 events.

Result. No events found in the last 4 hours.

Interpretation. P5 is confirmed, and the result is expected: the sample was only analysed in VirusTotal and was never executed on this VM. The hunt is still useful. It shows the intel-driven model (starting from an IOC), and it connects this week to Weeks 2 and 3. A negative IOC search

never proves a system is clean, because a new variant would have a different hash; this is why behaviour-based hunts such as Hunts 1–2 matter more.

7. Summary of Findings

Hunt	Question	Result	Meaning
1	Children of wsmprovhost.exe	5 events	Remote WinRM commands are traceable by their parent process. P1 confirmed.
2	Same tools, local vs remote	9 events (3 local + 6 remote)	Parent process separates the two cases. P2 confirmed.
3	3+ discovery tools in 5 min	2 clusters	Detects the remote run but also flags the local control: a false positive. P3 partly confirmed.
4	Connections to ports 5985/5986	10 events	Network traces match the timing of the remote commands. P4 confirmed.
5	Embargo hash on the host	0 events	Expected; the sample never ran here. P5 confirmed.

Hypothesis outcome. The hypothesis is **supported** in the lab: discovery commands sent over WinRM are started by wsmprovhost.exe, while the same commands run locally are started by powershell.exe, and Sysmon process-creation data is enough to tell them apart.

Link to earlier weeks. In Week 4 the detection recommendations included monitoring child processes of wsmprovhost.exe. Hunts 1 and 2 test that idea in practice. Week 2 supplied the hash for Hunt 5, and Week 3 supplied the habit of separating observation from proof, which we also applied here (for example, the false positive in Hunt 3 is reported openly).

8. Limitations and Improvements

- **Single machine.** The attacker and the victim are one VM, so the WinRM connection is a loopback and there is no remote source host or source account to examine.
- **Only Sysmon data.** We did not ingest the Windows Security log (logon event 4624, logon type 3), the PowerShell logs (events 4103/4104) or the WinRM Operational log (event 91). Week 4 recommended these sources, and adding them would give a second, independent view of the same activity.
- **Default Sysmon setup.** Sysmon ran with no tuned rule set ("No rules installed"). A production deployment would use a maintained configuration.
- **No baseline.** Administrators also create wsmprovhost.exe children in real life. A real hunt needs a baseline of normal remote administration so that expected activity can be excluded.
- **Small and artificial data.** Activity was created by us in a short time window, so the results show that the method works, not how often it would raise false alarms in production.
- **Possible improvement.** Turn Hunt 1 plus Hunt 3 into one alert: discovery tools with the parent wsmprovhost.exe, grouped by account and host, with an allow-list for known administrators.

9. Conclusion

In Week 5 we moved from collecting and organising threat intelligence to using it for detection. Starting from the Storm-0501 behaviour documented in Week 4, we formed a testable hypothesis, built a small lab with WinRM, Sysmon and Splunk, and ran five hunts. The main result is that the parent process wsmprovhost.exe reliably separates remote WinRM command execution from local PowerShell activity, and the local control run proved that this signal is specific to WinRM. The clustering hunt also showed that a rule can look good and still produce false positives, and the hash search showed the difference between intel-driven and behaviour-based hunting. Overall, the course sequence (OSINT, MISP, Kill Chain, hunting) now forms one chain: from a technique (T1021.006) to a concrete detection that can be tested in a log platform.

References

- Microsoft. Microsoft Threat Hunting Guide (recommended reading in the syllabus).
- Smith, P. Practical Threat Hunting (recommended reading in the syllabus).
- MITRE ATT&CK. Remote Services: Windows Remote Management (T1021.006). <https://attack.mitre.org/techniques/T1021/006/>
- MITRE ATT&CK. Storm-0501 (G1053). <https://attack.mitre.org/groups/G1053>
- Microsoft Threat Intelligence (2024, September 26). Storm-0501: Ransomware attacks expanding to hybrid cloud environments.
- Microsoft Threat Intelligence (2025, August 27). Storm-0501's evolving techniques lead to cloud-based ransomware.
- Microsoft Sysinternals. Sysmon documentation. <https://learn.microsoft.com/sysinternals/downloads/sysmon>
- Splunk Enterprise documentation. <https://docs.splunk.com>